trust graduation

Trust Graduation Protocol

GitHub, npm, protocol, schema, and roadmap documentation packet.

Trust Graduation

Trust Graduation is an open protocol for bounded agent authority.

It answers a narrower question than generic agent permissions:

> What has this agent earned the right to do for this principal, in this action class, under these constraints?

Trust Graduation does not grant global trust. It evaluates one requested action at a time using policy, evidence, approval semantics, and audit hooks.

Apache-licensed. Zero dependencies. Reference implementation: Mission by Phenomena Labs Ltd.

The long-form thinking is in MANIFESTO.md. The protocol docs live in docs/spec-overview.md and docs/spec-deep-dive.md.

Install

```
npm install @trust-graduation/core
```

Minimal Embed

```
import { TrustGraduation } from "@trust-graduation/core"; / const tg = new TrustGraduation({ workspace: "user-123", evidence: localLedger }); / const decision = tg.canExecute({ / actionClass: "email.send.external", / context: { / principal: "user-123", / requestedBy: "assistant", / recipient: "buyer@example.com", / body, / constraints: { scope: "once" } / } / }); / if (decision.allowed) await actuallySend(); / else if (decision.needsApproval) await pushApprovalToUser(decision.packet);
```

High-risk external actions remain approval-gated by default: sends, public posts, money movement, legal commitments, policy changes, and authority expansion.

Protocol Objects

Trust Graduation v0.1 centers on five objects:

- `ActionClassPolicy`
- `EvidenceEvent`
- `Decision`
- `ApprovalPacket`
- `ExecutionReceipt` as a forward-compatible hook for the separate receipts primitive

The runtime package emits `Decision` objects and bounded `ApprovalPacket` payloads now. Receipts remain a separate protocol primitive under active design.

Core Lifecycle

- An agent proposes a requested action in an action class.
- The host evaluates policy plus evidence.
- The host returns a `Decision`.
- If review is required, the host issues an `ApprovalPacket`.
- If a human approves, the bounded action may execute.
- Execution, outcomes, corrections, and rollbacks become future evidence.

What This Repo Contains

- `src/` — zero-dependency JavaScript reference implementation.
- `schemas/v1/` — JSON schemas for action classes, evidence, decisions, approval packets, and license entitlements.
- `schemas/v2/receipts.schema.json` — forward-design preview for the receipts primitive.
- `docs/spec-overview.md` — portable protocol overview.
- `docs/spec-deep-dive.md` — protocol objects, lifecycle, regression, and conformance guidance.
- `docs/receipts-forward-design.md` — storage-agnostic receipts direction.
- `docs/pdf/trust-graduation-protocol.pdf` — printable protocol packet generated with `npm run docs:pdf`.
- `examples/minimal.js` — minimal embed example.
- `packages/python/` and `packages/go/` — package placeholders for language ports.

Core Concepts

- Action class: the smallest portable unit of earned autonomy, such as `draft.response` or `email.send.external`.
- Evidence ledger: real approvals, edits, rejections, executions, receipts, outcomes, trust issues, and rollbacks.
- Autonomy level: the current earned capability for an action class.
- Approval packet: a portable bounded-review payload any product can render.
- Decision: the protocol object that explains whether the requested action is allowed now, gated, or regressed.
- License entitlement: an optional product/package capability gate. It does not grant autonomy.

License Entitlements

The package defaults to a free local protocol license. Tokens are intentionally simple in alpha: `tg1.<base64url-json>`.

They do not contact a server and they do not change Trust Graduation decisions.

```
import { createLicenseToken, decodeLicenseToken, licenseAllows } from "@trust-graduation/core"; / const token =
createLicenseToken({ subject: "workspace-123", features: ["core", "schemas"] }); / const status =
decodeLicenseToken(token); / if (licenseAllows(status, "core")) { / // run the local reference implementation / }
```

Future hosted federation or enterprise support can issue stronger tokens without changing how an embedder calls `canExecute()`.

Status

Package status: `0.1.0-alpha.3`.

Schema status: draft `schemas/v1/`.

Schemas describe the protocol shape. The package is the JavaScript reference implementation. Schemas reach stable `v1.0` after outside implementations validate the object model and no breaking schema changes are required for a sustained period.

License

Apache-2.0.

Trust Graduation Protocol v0.1 Overview

Status: alpha draft

Purpose: define a portable protocol for bounded agent authority

Trust Graduation is an open protocol for deciding what an agent has earned the right to do for a principal, in a specific action class, under explicit constraints, with evidence, approval semantics, and auditability.

The protocol does not grant global trust. It evaluates one requested action at a time against policy, evidence, approvals, and rollback expectations.

Core Sentence

An agent may execute only what it has earned for this principal, in this action class, under these constraints.

Design Goals

- Bound authority to one action class at a time.
- Separate policy, evidence, decision, approval, execution, and receipt.
- Keep high-risk external actions approval-gated by default.
- Make trust regressible when evidence deteriorates.
- Stay portable across local runtimes, HTTP APIs, MCP surfaces, and multi-agent systems.
- Let products adopt the protocol without adopting Mission.

Core Objects

Trust Graduation v0.1 defines five primary protocol objects:

- `ActionClassPolicy`
- `EvidenceEvent`
- `Decision`
- `ApprovalPacket`
- `ExecutionReceipt` (forward-compatible in this repo; the full receipts primitive remains separate)

The reference implementation also defines a license entitlement payload. Entitlements govern package or service features. They do not grant autonomy and they do not change decision outcomes.

Core Lifecycle

- An agent proposes a requested action in an action class.
- The host evaluates the request against policy and evidence.

- The host returns a `Decision`.
- If review is needed, the host issues an `ApprovalPacket`.
- If a human approves, the bounded action may execute.
- Execution produces an audit trail and, where applicable, a receipt.
- Outcomes, corrections, violations, and rollbacks become future `EvidenceEvent` rows.

Autonomy Ladder

Level	Name	Meaning
0	Observe	Read, summarize, classify, flag, and explain.
1	Prepare	Draft, rank, route, and assemble review material.
2	Stage	Pre-fill bounded local actions or ready fields for final review.
3	Execute Narrow	Execute low-risk, reversible actions in a bounded lane with receipts.
4	Delegate	Run a bounded workflow inside a defined domain with review hooks.
5	Govern	Recommend policy or autonomy changes, never self-approve them.

Autonomy is not a blanket product setting. Each action class has its own current level, evidence history, and regression state.

Default Boundary

The following remain approval-gated by default:

- external sends
- public posting
- money movement
- legal commitments
- irreversible submissions
- permission changes
- policy changes
- authority expansion

A product may define narrower policies, but it should not silently weaken these defaults.

Required Trust Properties

A conforming implementation should make the following visible for each requested action:

- who is acting
- for whom they are acting

- what action class is requested
- what constraints apply
- what evidence was considered
- whether approval is required
- whether the action executed
- what receipt or audit record exists
- how the trust state can regress

Public Promise

Never sends without approval.

Trust Graduation Protocol v0.1 Deep Dive

Status: alpha draft

Purpose: define the protocol surface independent of any one product

1. Scope

Trust Graduation is a protocol for bounded agent authority.

It answers a narrower question than general authorization systems:

> Has this agent earned the right to perform this class of action for this principal under the current constraints?

The protocol is intentionally decision-centric. It does not require a specific storage engine, identity provider, user interface, transport, or machine learning model.

2. Actors

A conforming implementation should model these roles explicitly even if one product collapses them internally:

- `principal`: the person or organization whose trust boundary is being protected
- `agent`: the software actor proposing or taking action
- `host`: the product or runtime evaluating Trust Graduation rules
- `approver`: the human or delegated authority who can approve a bounded action
- `executor`: the component that performs the side effect once allowed
- `auditor`: a human or system reviewing receipts, violations, and regressions

3. Action Classes

An action class is the smallest portable unit of earned autonomy.

Examples:

- `draft.response`
- `calendar.propose.internal`
- `calendar.send.external`
- `email.send.external`
- `task.cleanup.local`
- `policy.change`

Action classes should be:

- stable enough to accumulate evidence over time
- narrow enough that evidence on one class does not over-grant another
- legible to users, auditors, and integrators

An implementation may normalize product-specific actions into a shared action-class vocabulary.

4. Protocol Objects

4.1 `ActionClassPolicy`

Defines the default trust boundary for one action class.

Recommended fields:

- `actionClass`
- `description`
- `riskClass`
- `minimumLevel`
- `requiresApproval`
- `receiptRequired`
- `externalSideEffects`
- `reversible`
- `defaultConstraints`
- `regressionTriggers`

4.2 `EvidenceEvent`

Represents one trust-relevant event for one action class.

Required semantic fields:

- `actionClass`
- `type`

Recommended fields:

- `eventId`
- `principal`
- `agent`
- `source`
- `recordedAt`
- `decisionId`
- `approvalPacketId`

- `receiptId`
- `outcomeId`
- `editDistance`
- `severity`
- `metadata`

Canonical event types should include at least:

- `approved`
- `edited`
- `rejected`
- `held`
- `executed`
- `receipt_logged`
- `outcome_positive`
- `outcome_negative`
- `trust_issue`
- `rollback`
- `policy_exception`

Evidence must come from real interaction, execution, outcome, or repair. More drafts, raw model confidence, or synthetic success claims are not evidence.

4.3 `Decision`

Represents the output of evaluating one requested action.

A `Decision` should answer:

- is the action allowed now?
- is approval required?
- what mode applies?
- what trust state was used?
- what policy justified it?
- what evidence summary justified it?
- what approval packet or next step is required?

Recommended fields:

- `decisionId`
- `requestedAction`
- `actionClass`
- `allowed`

- `needsApproval`
- `mode`
- `autonomyLevel`
- `tier`
- `policy`
- `evidence`
- `constraints`
- `reason`
- `packet`
- `createdAt`

4.4 `ApprovalPacket`

A portable payload for bounded human review.

A conforming packet should make the human able to answer:

- what exactly is being asked?
- why is it gated?
- what risk class applies?
- what evidence exists?
- what can I approve once versus reject or revise?
- what receipt will exist if this runs?

Recommended fields:

- `packetId`
- `decisionId`
- `workspace` or `scope`
- `requestedBy`
- `principal`
- `actionClass`
- `riskClass`
- `externalSideEffects`
- `approvalRequired`
- `receiptRequired`
- `reason`
- `requestedAction`
- `constraints`
- `evidence`
- `decisions`

- `createdAt`
- `expiresAt`

4.5 `ExecutionReceipt`

The full receipts primitive is still separate, but implementations should already assume the need for a portable execution record.

Minimum expectations:

- a stable `receiptId`
- linkage to the decision or approval packet
- summary of what executed
- time of execution
- executor identity when available
- target summary
- rollback pointer or reason if repaired later

5. Evidence Tiers And Autonomy

This reference implementation uses four evidence tiers:

Tier	Meaning
`gated`	Default state or insufficient evidence
`supervised`	Positive evidence exists, but review boundaries still matter
`auto_capped`	Repeated clean evidence for narrow bounded automation
`review`	Negative evidence or trust regression requires explicit review

These tiers are implementation guidance, not the entire protocol. A host may use different internals as long as it returns a conforming `Decision` and preserves the core trust boundary.

6. Decision Modes

A conforming implementation should use explicit decision modes rather than a single boolean.

Recommended modes:

- `allowed`
- `supervised`
- `auto_capped`
- `approval_required`

- `review_only`
- `insufficient_evidence`
- `denied`
- `approved_once`

This makes the protocol legible to products, logs, and auditors.

7. Constraints

Trust Graduation is only meaningful if approvals and grants are bounded.

Constraints may include:

- one-time execution
- allowed recipients or domains
- amount or consequence caps
- time limits or expiry
- required human-visible fields
- rollback expectation
- disallowed data sources
- no execution from untrusted retrieved content alone

A request should never be treated as globally approved just because a similar action was approved before.

8. Regression

Trust must be able to move down.

Regression should occur when an action class accumulates events such as:

- trust issues
- repeated rejections
- high edit distance after prior confidence
- rollback events
- unsupported claims
- wrong recipient or target selection
- policy exceptions
- negative downstream outcomes

A protocol implementation should expose regression visibly through the returned `Decision` or associated audit state.

9. Conformance Requirements For v0.1

A v0.1 conforming implementation should:

- evaluate trust per action class, not globally
- expose a decision artifact, not only UI behavior
- preserve an evidence model with real event types
- support bounded approval packets for gated actions
- keep high-risk external side effects approval-gated by default
- support regression after negative evidence
- make execution auditable with a stable identifier or receipt hook

A v0.1 conforming implementation does not need to:

- use cryptographic signatures
- use decentralized identity
- use a hosted service
- share evidence across products
- use Mission’s exact heuristics or thresholds

10. Extensions

The following fit naturally as protocol extensions rather than core requirements:

- signed receipts
- verifiable identity for principal, agent, and approver
- delegated actor chains
- cross-product evidence portability
- consented federation
- domain-specific action-class registries

These should layer on top of the core decision contract, not replace it.

Receipts Forward Design

Trust Graduation core decides what an agent has earned the right to do. Receipts are the evidence that something actually happened.

This document is forward design, not a shipped package. `@trust-graduation/receipts` should only ship after `@trust-graduation/core` has at least one external embed.

Why Receipts Exist

Every agent product needs an audit trail. Without a shared receipt shape, each agent keeps private logs that other agents cannot use. A Trust Graduation receipt makes completed work legible across products:

- Core can treat receipts as evidence for autonomy graduation.
- Other agents can avoid duplicating work that has already closed.
- Users get one audit trail for agent-mediated actions.
- Future federation can share evidence across products with explicit consent.

Minimal Shape

```
{ / "protocol": "trust-graduation-receipts", / "version": "1.0", / "receiptId": "rcpt_20260531_abc", / "workspace": "user-ronen", / "agent": "cursor", / "actionClass": "code.commit", / "result": "completed", / "evidence": { / "target": "commit:abc123", / "summary": "Added auth middleware", / "approvedBy": "human:ronen", / "approvedAt": "2026-05-31T10:00:00Z", / "completedAt": "2026-05-31T10:00:30Z", / "humanEditDistance": 0.05, / "outcome": "merged" / }, / "links": { / "approvalPacketId": "tg_xyz", / "openLoopId": "ol_abc", / "ruleEvidence": ["rule:proof-backed-followups"] / } / }
```

Package Sketch

```
import { ReceiptStore } from "@trust-graduation/receipts"; / const store = new ReceiptStore({ workspace: "user-ronen" }); / const receipt = store.append({ / agent: "my-agent", / actionClass: "email.send.external", / result: "completed", / evidence: { / target: "msg:xyz", / summary: "Sent approved follow-up" / } / }); / const recent = store.query({ / actionClass: "email.send.external", / since: "2026-05-24T00:00:00Z", / federated: true / });
```

The package should stay storage-agnostic: local file, product database, and `trust-graduation.org/api` federation can all satisfy the same interface.

Discipline Gate

Do not ship `@trust-graduation/receipts` until:

- `@trust-graduation/core` has one external embed.
- Mission emits receipts internally in a way that maps cleanly to the schema.
- One external partner wants receipt emission or consumption enough to review the schema.

Until then, the only artifact is the schema preview at `schemas/v2/receipts.schema.json`.

Protocol Adoption Roadmap

Status: alpha operating plan

Phase 1: Crystallize

Goal: a public spec and package any agent vendor can embed in less than 100 lines.

- Keep the reference implementation pure: JSON in, JSON out.
- Publish the npm package first.
- Keep Python and Go ports API-compatible with the JavaScript decision contract.
- Publish schemas under `/schemas/v1/`.
- Keep license entitlement tokens free-local by default; use them to prove the future monetization boundary without blocking adoption.
- Use the Manifesto as launch positioning, not as a substitute for an embeddable package.

Phase 2: Seed Adoption

Goal: three named external integrations, including one credible agent product or framework.

Pitch:

> We will help you embed Trust Graduation in one day. You get an approval/evidence safety layer and a standard audit trail. We get the citation.

Good first targets:

- agent frameworks with active maintainers
- open-source products where a PR can be reviewed publicly
- AI infra teams already wrestling with autonomy gates

Phase 3: Federation

Goal: opt-in cross-product evidence sharing.

The alpha package does not implement federation. It is shaped so a hosted service can later provide the same evidence ledger from multiple products with user consent.

Phase 4: Governance

Goal: move from a Mission-originated reference implementation to a citeable standard.

Governance should remain lightweight until there are real adopters. Do not create process before adoption.

Phase 5: Monetization

Potential protocol-side revenue:

- hosted federation
- compliance/audit pack
- enterprise support contracts
- white-label implementation support

The first monetization gate is not pricing. It is external adoption.

In alpha, entitlement tokens should stay permissive. The purpose is to make future hosted federation, compliance packs, and enterprise support attach cleanly to protocol features after adoption, not to add friction before the standard has citations.

GitHub And npm Publishing Guide

Trust Graduation public artifacts should make the protocol easy to inspect, install, and cite.

What Goes To GitHub

The GitHub repo is the public protocol home. It should include:

- `README.md` - hook, install, six-line embed, status, roadmap, and links.
- `src/` - zero-dependency JavaScript reference implementation for `@trust-graduation/core`.
- `schemas/v1/` - stable alpha schemas for core autonomy primitives.
- `schemas/v2/receipts.schema.json` - forward-design preview for receipts, not a shipped package.
- `docs/spec-overview.md` - one-page spec overview.
- `docs/spec-deep-dive.md` - compact protocol details.
- `docs/receipts-forward-design.md` - next-pillar design note.
- `docs/adoption-roadmap.md` - external embed path and discipline gates.
- `docs/github-npm-publishing.md` - this publishing map.
- `docs/pdf/trust-graduation-protocol.pdf` - printable protocol packet.
- `examples/minimal.js` - six-line embed example.
- `packages/python/README.md` and `packages/go/README.md` - language-port placeholders until parity work begins.
- `CONTRIBUTING.md`, `CHANGELOG.md`, `LICENSE`.

The repo should not include private Mission workspace data, customer context, local receipts, secrets, unpublished GTM notes, or product-specific Mission Control internals.

What Goes To npm

The npm package is the embeddable runtime. It should include:

- `src/` - runtime implementation and TypeScript declarations.
- `schemas/` - protocol schemas.
- `docs/` - public docs and PDF protocol packet.
- `examples/` - minimal embed examples.
- `packages/*/README.md` - language-port status.
- `README.md`, `LICENSE`.

The npm package should not include:

- local workspace data
- `.git`
- private strategy notes

- generated logs
- secrets or connector credentials
- unpublished partner/customer context

The current package boundary is controlled by `package.json` `files`.

Release Checklist

Before publishing:

- Run `npm test`.
- Run `npm run docs:pdf`.
- Verify `README.md` install and import snippets use `@trust-graduation/core`.
- Verify `docs/pdf/trust-graduation-protocol.pdf` exists and renders.
- Verify `package.json` version and `CHANGELOG.md` agree.
- Run `npm pack --dry-run` and inspect included files.
- Publish only after the GitHub repo is pushed.

Discipline Gate

Do not ship `@trust-graduation/receipts`, `@trust-graduation/open-loops`, `@trust-graduation/voice`, or federation packages until the prior primitive has external embed traction. Forward-design schemas are allowed; runtime packages wait for demand.